UNIVERSIDAD ESTATAL PENINSULA DE SANTA ELENA

SOLUCIÓN DE BUSINESS INTELLIGENCE DE EXTREMO A EXTREMO

ESTUDIANTE:
ANDY WILFRIDO QUIMI QUIMI

MATERIA:
INTELIGENCIA DE NEGOCIOS

DOCENTE:
ING. ANTHONY PACHAY ESPINOZA, MSC

CURSO:
5

PARALELO:
1

REPOSITORIO:
Andyquimi/bi-olist-entregable3: Solución de BI con PostgreSQL, Python y Streamlit (dataset Olist)

LINK DEL VIDEO:
https://drive.google.com/file/d/1a5yoY9rqpREGdJgoE0TCxN9wZE7lzNjU/view?usp=sharing

AÑO:
2026-2027

1. Dataset y comprensión del problema

1.1 Ficha del dataset

Campo	Detalle
Nombre	Brazilian E-Commerce Public Dataset by Olist
Autor / fuente	Olist (marketplace brasileño), publicado en Kaggle
Enlace de origen	https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce
Descripción general	Datos reales y anonimizados de pedidos realizados en la tienda Olist y en varios marketplaces de Brasil. Permite ver cada pedido desde múltiples ángulos: estado, precio, pago, flete, ubicación del cliente, atributos del producto y reseña escrita por el cliente.
Estructura	9 archivos CSV relacionales (pedidos, ítems, pagos, reseñas, clientes, vendedores, productos, geolocalización y traducción de categorías).
Registros	99 441 pedidos y 112 650 ítems vendidos; el archivo más grande es geolocalización (aprox. 1 millón de filas).
Columnas	52 columnas en total entre las 9 tablas (de 2 a 9 por tabla).
Período	Compras entre 2016 y 2018. Se analiza de enero de 2017 a agosto de 2018 (2016 y septiembre-octubre de 2018 tienen muy pocos registros).
Licencia	CC BY-NC-SA 4.0 (uso público en Kaggle).

1.2 Archivos que componen el dataset

Archivo CSV	Contenido	Filas	Cols.
olist_orders	Pedidos: cliente, estado y fechas del proceso (núcleo)	99 441	8
olist_order_items	Ítems de cada pedido: producto, vendedor, precio y flete	112 650	7
olist_order_payments	Formas y montos de pago	103 886	5
olist_order_reviews	Calificación (1-5) y comentario del cliente	99 224	7
olist_customers	Cliente por pedido y su ubicación	99 441	5
olist_sellers	Vendedores y su ubicación	3 095	4
olist_products	Atributos del producto: categoría, peso, dimensiones	32 951	9
olist_geolocation	Código postal con latitud y longitud	1 000 163	5
product_category_name_translation	Traducción de categorías (portugués a inglés)	71	2

1.3 Qué representa una fila

Como el dataset está normalizado, la fila significa algo distinto en cada tabla. La granularidad más fina, y la que se usa para el análisis, es el ítem del pedido.

Tabla	Una fila representa
orders	Un pedido realizado por un cliente, con su estado y las fechas de compra, aprobación, envío, entrega y entrega estimada.
order_items	Un producto vendido dentro de un pedido (un pedido con 3 artículos genera 3 filas). Es la fila de mayor detalle y la base de la tabla de hechos.
order_payments	Un pago parcial de un pedido (puede pagarse con varios métodos o cuotas).
order_reviews	Una reseña enviada por el cliente sobre un pedido.
customers	Un registro de cliente por pedido: customer_id cambia en cada compra; customer_unique_id identifica a la persona real.
sellers / products	Un vendedor / un producto del catálogo.
geolocation	Un punto de coordenadas asociado a un prefijo de código postal (con muchas repeticiones).

1.4 Variables principales

Tipo	Variables principales	Uso analítico
Identificadores	order_id, customer_id, customer_unique_id, seller_id, product_id, review_id, order_item_id	Claves para unir tablas y contar entidades; se convierten en llaves de dimensiones.
Numéricas	price, freight_value, payment_value, payment_installments, review_score, product_weight_g, product_length/height/width_cm	Medidas (ingresos, flete, pagos) y atributos del producto; se agregan con suma, promedio o conteo.
Categóricas	product_category_name, payment_type, customer_city/state, seller_city/state	Ejes de segmentación: categoría, método de pago y ubicación.
Temporales	order_purchase_timestamp, order_approved_at, order_delivered_carrier_date, order_delivered_customer_date, order_estimated_delivery_date, review_creation_date	Tendencia, estacionalidad, tiempos de entrega y retrasos.
Estados / resultados	order_status (delivered, shipped, canceled, unavailable, invoiced, processing, created, approved), review_score	Resultado del pedido y satisfacción; sirven para filtrar pedidos válidos.
Texto libre	review_comment_title, review_comment_message	Posible análisis de sentimiento (fuera del alcance).

1.5 Justificación de la selección

Antes de elegirlo se evaluó si el dataset permite desarrollar el proyecto completo de BI. Se seleccionó por las siguientes razones:

- **Modelo de negocio completo:** cubre todo el ciclo de venta (compra, pago, envío, entrega y reseña), por lo que admite análisis comerciales, logísticos y de satisfacción.
- **Normalizado en varias tablas:** obliga a un proceso ETL real (uniones, limpieza, transformación) y justifica un modelo dimensional.
- **Dimensiones naturales:** tiempo, cliente, vendedor, producto, ubicación geográfica y estado del pedido.
- **Volumen adecuado:** cerca de 100 mil pedidos son suficientes para que un almacén de datos y un dashboard tengan sentido, sin exceder la capacidad de un equipo personal.
- **Calidad de datos imperfecta y realista:** nulos, duplicados y categorías en portugués permiten demostrar el trabajo de limpieza.
- **Multivariable:** combina medidas numéricas, fechas, texto y coordenadas, lo que abre la puerta a mapas, series de tiempo y análisis de satisfacción.

1.6 Problema o necesidad de análisis

Un marketplace necesita saber cómo evolucionan sus ventas, si cumple los plazos de entrega que promete y qué vendedores y categorías sostienen sus ingresos y su reputación. Hoy esa información está dispersa en 9 tablas relacionales difíciles de consultar. La solución centraliza los datos en un modelo dimensional en PostgreSQL y los presenta en un dashboard interactivo.

1.7 Problemas de calidad identificados

Tipo	Problema detectado	Tratamiento aplicado
Valores nulos	orders: fechas de aprobación, envío y entrega vacías en pedidos no completados. reviews: título vacío en cerca del 88 % y mensaje en cerca del 59 %. products: 610 sin categoría.	Se mantienen nulos en fechas y los tiempos se calculan solo con pedidos entregados. Categoría faltante = "sin_categoria". Se excluyen los comentarios de texto.
Duplicados	geolocation: cientos de miles de filas repetidas por código postal. reviews: review_id repetido en algunos casos.	Un punto por código postal (promedio de coordenadas). Se conserva la reseña más reciente por pedido.
Tipos incorrectos	Todas las fechas se cargan como texto; los códigos postales pueden perder ceros iniciales.	Conversión a datetime y lectura de códigos postales como texto.
Valores atípicos	price y freight_value con máximos muy superiores al promedio; pagos con valor 0; fechas de entrega anteriores a la compra.	Análisis con boxplot/IQR (8 388 precios atípicos): se conservan por ser precios altos legítimos. Se descartan fechas incoherentes (0 casos).
Categorías inconsistentes	Categorías en portugués con guiones bajos; ciudades con acentos y grafías distintas; tipo de pago "not_defined".	Unión con la tabla de traducción; ciudades normalizadas (minúsculas, sin acentos).
Estructura confusa	customer_id cambia en cada pedido; columnas con errores ortográficos (product_name_lenght).	Se usa customer_unique_id para identificar clientes; columnas renombradas en el ETL.

Tipo	Problema detectado	Tratamiento aplicado
Columnas poco útiles	review_comment_title/message, product_name_lenght, product_description_lenght, product_photos_qty.	Excluidas del modelo inicial.
Cobertura temporal	2016 y el cierre de 2018 casi no tienen datos y distorsionan las tendencias.	Se analiza el período enero 2017 – agosto 2018.

1.8 Usuarios potenciales y decisiones

Usuario	Qué consulta	Decisiones que puede tomar
Gerente comercial	Ingresos, ticket promedio, categorías y regiones	Definir promociones, ampliar el catálogo de las categorías con mejor desempeño y priorizar estados con alta demanda.
Responsable de logística	Tiempos de entrega y retrasos por estado y vendedor	Renegociar con operadores logísticos, ajustar plazos estimados y mejorar rutas a las regiones lentas.
Gestor de vendedores	Ingresos y reputación por vendedor	Premiar a vendedores destacados y advertir o sancionar a los de mala calificación o alto retraso.
Atención al cliente / calidad	Calificaciones y su relación con los retrasos	Intervenir de forma anticipada en pedidos con riesgo de mala reseña.
Marketing / dirección	Recompra y medios de pago	Diseñar campañas de fidelización y promociones por método de pago o cuotas.

2. Preguntas de negocio e indicadores

2.1 Preguntas de negocio

#	Pregunta de negocio	Tablas del modelo
P1	¿Cómo evolucionan las ventas por mes y qué categorías de producto y estados generan más ingresos?	fact, dim_fecha, dim_producto, dim_cliente
P2	¿Qué tan bien cumple Olist con los plazos de entrega y qué estados o vendedores presentan más retrasos?	fact, dim_cliente, dim_vendedor
P3	¿Cómo afecta el retraso en la entrega a la satisfacción del cliente (calificación de la reseña)?	fact (dias_retraso, review_score)
P4	¿Qué vendedores concentran los ingresos y mantienen mejor reputación (calificación promedio)?	fact, dim_vendedor
P5	¿Qué medios de pago prefieren los clientes y cuántos regresan a comprar?	fact, dim_pago, dim_cliente

2.2 KPI definidos

Cada KPI se calcula realmente desde el modelo dimensional implementado, mediante una función SQL.

KPI	Definición / fórmula	Función SQL	Resultado (todo el período)
Ingresos por ventas	SUM(price) de ítems de pedidos válidos (se excluyen cancelados y no disponibles)	fn_total_ventas()	R$ 13 449 529.68
Ticket promedio (AOV)	Ingresos / número de pedidos únicos	fn_ticket_promedio()	R$ 137.37
Tasa de entrega a tiempo	Pedidos entregados con fecha real ≤ estimada / pedidos entregados × 100	fn_tasa_entrega_a_tiempo()	93.21 %
Tiempo promedio de entrega	AVG(fecha de entrega al cliente − fecha de compra), en días	fn_tiempo_promedio_entrega()	12.5 días
Calificación promedio	AVG(review_score) por pedido	fn_calificacion_promedio()	4.12 / 5
Tasa de recompra	Clientes únicos con más de un pedido / clientes únicos totales × 100	fn_tasa_recompra()	3.03 %

3. Limpieza y transformación de datos

El script python/etl_olist.py aplica 13 pasos de limpieza con pandas antes de cargar los datos al modelo. La tabla resume cada paso y su efecto en el número de filas.

Paso	Tabla	Filas antes	Filas después	Transformación
1	orders	99 441	99 441	Fechas convertidas de texto a datetime
2	orders	99 441	99 441	Eliminados order_id duplicados
3	orders	99 441	99 092	Período de análisis: 2017-01-01 a 2018-08-31 (se descarta 2016 y sep-oct 2018)
4	orders	99 092	99 092	Descartados pedidos con fecha de entrega ANTERIOR a la compra
5	orders	99 092	99 092	Estado normalizado (minúsculas); fechas nulas se mantienen en pedidos no entregados
6	items	112 650	112 279	Solo ítems de pedidos del período; fecha convertida a datetime
7	items	112 279	112 279	Valores atípicos de price detectados (boxplot/IQR): 8,388. Se CONSERVAN (son precios altos legítimos)
8	customers	99 441	99 092	Solo clientes con pedidos; ciudad normalizada (minúsculas, sin acentos); zip como texto
9	sellers	3 095	3 095	Ciudad normalizada; zip como texto
10	products	32 951	32 951	Unido con traducción de categorías; 610 sin categoría -> 'sin_categoria'; 0 sin traducción; columnas renombradas (se corrigen errores ortográficos)
11	payments	103 886	99 092	Pagos resumidos a UNA fila por pedido (tipo principal = el de mayor valor; cuotas = máximo)
12	reviews	99 224	98 330	Una reseña por pedido (la más reciente); comentarios de texto excluidos (88 % / 59 % vacíos)
13	geoloc	1 000 163	19 010	De-duplicada: UN punto por código postal (promedio de coordenadas)

Variables derivadas (se calculan al cargar la tabla de hechos, sql/03): dias_entrega (fecha de entrega al cliente − fecha de compra), dias_retraso (días de retraso respecto a la fecha estimada), entrega_a_tiempo (1 si la entrega fue en la fecha estimada o antes, 0 si no) y región del cliente a partir de su estado.

```
PS C:\Users\Usuario\Downloads\Entregable3_BI_Olist\entregable3> docker compose run --rm etl
[+] run 1/1
 ✔Container bi-postgres Running                                          0.0s
Container bi-postgres Waiting
Container bi-postgres Healthy
Container bi-olist-etl-run-955dca10b771 Creating
Container bi-olist-etl-run-955dca10b771 Created
=== 0/7 Verificando CSV originales en data/raw ===
=== 1/7 Esperando a PostgreSQL (db:5432) ===
=== 2/7 Limpieza + carga a staging (python/etl_olist.py) ===
[ 1] orders        99,441 ->    99,441  Fechas convertidas de texto a datetime
[ 2] orders        99,441 ->    99,441  Eliminados order_id duplicados
[ 3] orders        99,441 ->    99,092  Período de análisis: 2017-01-01 a 2018-08-31 (se descarta 2016 y sep-oct 2018)
[ 4] orders        99,092 ->    99,092  Descartados pedidos con fecha de entrega ANTERIOR a la compra
[ 5] orders        99,092 ->    99,092  Estado normalizado (minúsculas); fechas nulas se mantienen en pedidos no entregados
[ 6] items        112,650 ->   112,279  Solo ítems de pedidos del período; fecha convertida a datetime
[ 7] items        112,279 ->   112,279  Valores atípicos de price detectados (boxplot/IQR): 8,388. Se CONSERVAN (son precios altos legítimos)
[ 8] customers     99,441 ->    99,092  Solo clientes con pedidos; ciudad normalizada (minúsculas, sin acentos); zip como texto
[ 9] sellers        3,095 ->     3,095  Ciudad normalizada; zip como texto
[10] products      32,951 ->    32,951  Unido con traducción de categorías; 610 sin categoría -> 'sin_categoria'; 0 sin traducción; columnas renombradas (se
 corrigen errores ortográficos)
[11] payments     103,886 ->    99,092  Pagos resumidos a UNA fila por pedido (tipo principal = el de mayor valor; cuotas = máximo)
[12] reviews       99,224 ->    98,330  Una reseña por pedido (la más reciente); comentarios de texto excluidos (88 % / 59 % vacios)
[13] geoloc     1,000,163 ->    19,010  De-duplicada: UN punto por código postal (promedio de coordenadas)

CSV limpios guardados en /app/data/limpio
Log de transformaciones en /app/docs/log_transformaciones.csv

Conectando a PostgreSQL ...
  stg.pedidos             99,092 filas cargadas
  stg.items              112,279 filas cargadas
  stg.clientes            99,092 filas cargadas
  stg.vendedores           3,095 filas cargadas
  stg.productos           32,951 filas cargadas
  stg.pagos_pedido        99,092 filas cargadas
  stg.resenas             98,330 filas cargadas
  stg.geolocalizacion     19,010 filas cargadas
Staging cargado correctamente.
=== 3/7 Modelo dimensional (hechos, dimensiones, PK y FK) ===
psql:sql/02_modelo_dimensional.sql:10: NOTICE:  schema "dw" already exists, skipping
```

```
psql:sql/02_modelo_dimensional.sql:10: NOTICE:  schema "dw" already exists, skipping
psql:sql/02_modelo_dimensional.sql:13: NOTICE:  drop cascades to 5 other objects
DETAIL:  drop cascades to view dw.v_logistica_estado
drop cascades to view dw.v_ventas_categoria
drop cascades to view dw.v_ventas_estado
drop cascades to view dw.v_metodos_pago
drop cascades to view dw.v_detalle_ventas
=== 4/7 Carga de dimensiones y tabla de hechos ===
=== 5/7 Validaciones ===
    -> evidencias/validaciones.txt
=== 6/7 Vistas, funciones y pruebas ===
psql:sql/05_vistas_funciones.sql:10: NOTICE:  schema "dw" already exists, skipping
psql:sql/05_vistas_funciones.sql:14: NOTICE:  drop cascades to 3 other objects
DETAIL:  drop cascades to view dw.v_pedidos
drop cascades to view dw.v_ventas_mensual
drop cascades to view dw.v_retraso_vs_resena
    -> evidencias/pruebas_objetos.txt
=== 7/7 Funciones de la app, validaciones extra y pruebas (Entregable 3) ===
    -> evidencias/validaciones_extra.txt y evidencias/pruebas_funciones_app.txt

LISTO. Conectate desde tu PC con: host=localhost  puerto=<DB_PORT, por defecto 5432>  bd=bi_database  usuario=bi_user
PS C:\Users\Usuario\Downloads\Entregable3_BI_Olist\entregable3>
```

Figura 1. Salida del ETL: tablas de staging cargadas con su número de filas

4. Modelo dimensional

4.1 Granularidad

Una fila de la tabla de hechos representa un ítem (producto) vendido dentro de un pedido.

Se elige este nivel porque es el más detallado y permite agregar hacia pedidos, productos, vendedores y períodos sin perder información.

4.2 Diagrama estrella

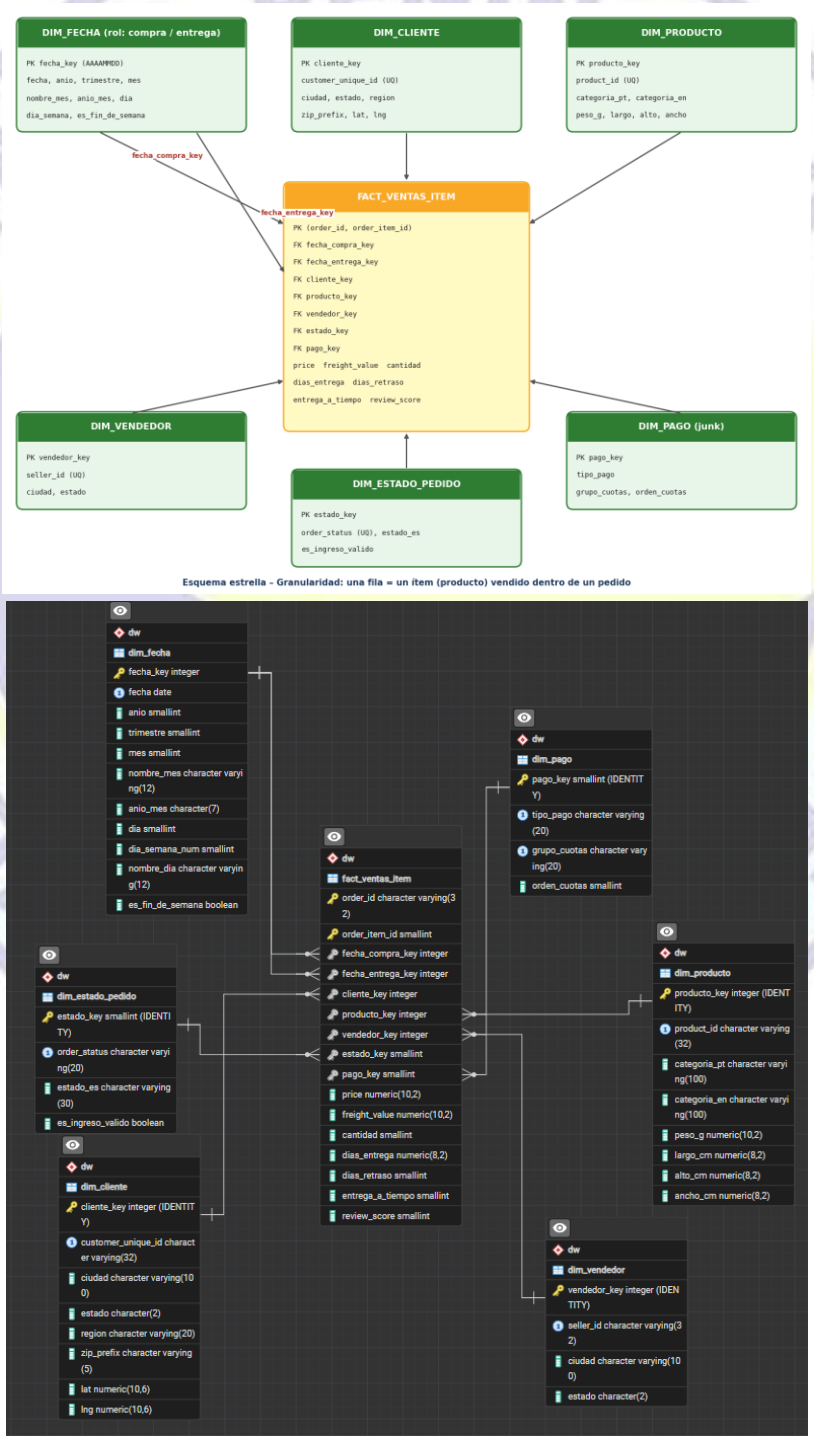


Figura 2. Esquema estrella del modelo dimensional

4.3 Tabla de hechos

Elemento	Detalle
Tabla	dw.fact_ventas_item (112 279 filas)
Clave primaria	Compuesta: (order_id, order_item_id). order_id es una dimensión degenerada.
Claves foráneas (7)	fecha_compra_key, fecha_entrega_key (ambas hacia dim_fecha), cliente_key, producto_key, vendedor_key, estado_key, pago_key
Medidas aditivas	price (ingreso), freight_value (flete), cantidad (= 1 por fila)
Medidas derivadas / semiaditivas	dias_entrega, dias_retraso, entrega_a_tiempo (0/1) y review_score; se promedian, no se suman. Son de nivel pedido y se agregan por pedido antes de promediar para no duplicar valores.

4.4 Dimensiones

Dimensión	Filas	Atributos	Jerarquía
dim_fecha (roles: compra y entrega)	1 461	fecha_key (AAAAMMDD), fecha, año, trimestre, mes, nombre del mes, año-mes, día, día de la semana, fin de semana	Día – Mes – Trimestre – Año
dim_cliente	95 774	customer_unique_id, ciudad, estado, región, zip_prefix, lat, lng	Ciudad – Estado – Región
dim_producto	32 951	product_id, categoría en portugués e inglés, peso, largo, alto, ancho	Producto – Categoría
dim_vendedor	3 095	seller_id, ciudad, estado	Ciudad – Estado
dim_estado_pedido	8	order_status, estado en español, es_ingreso_valido	—
dim_pago (dimensión basura)	25	tipo de pago principal, grupo de cuotas, orden de cuotas	—

4.5 Decisiones de diseño

- Pagos y reseñas están a nivel de pedido, no de ítem: se resumen a una fila por pedido en el ETL y los KPI se agregan por pedido antes de promediar, para no duplicar montos ni calificaciones.

La dimensión de fecha juega dos roles (fecha de compra y fecha de entrega) mediante dos claves foráneas.

dim_pago es una dimensión basura que agrupa el tipo de pago principal y el grupo de cuotas.

No se creó una segunda tabla de hechos a nivel de pedido: el tratamiento anterior resolvió el problema de la unión por ítem.

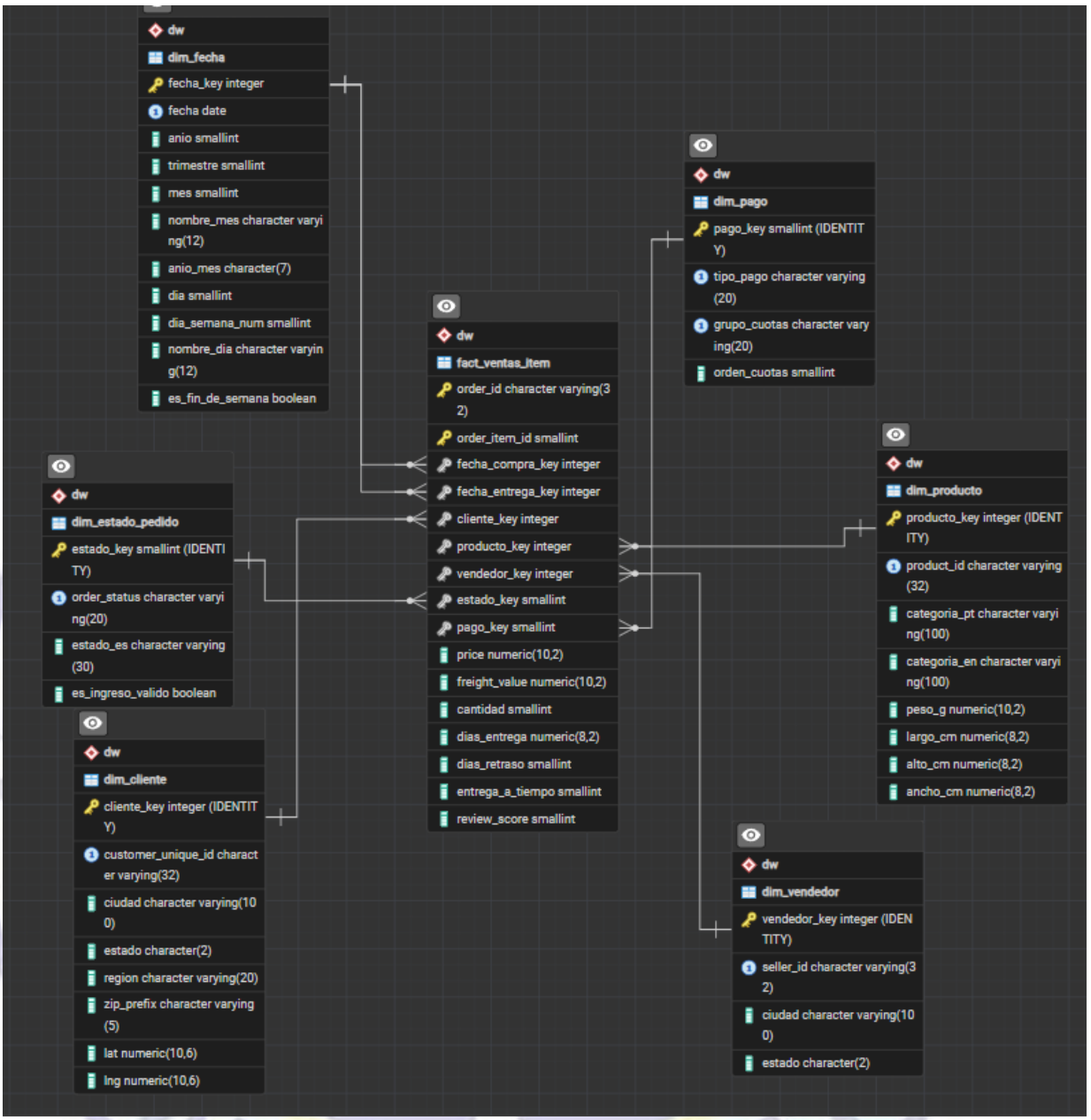
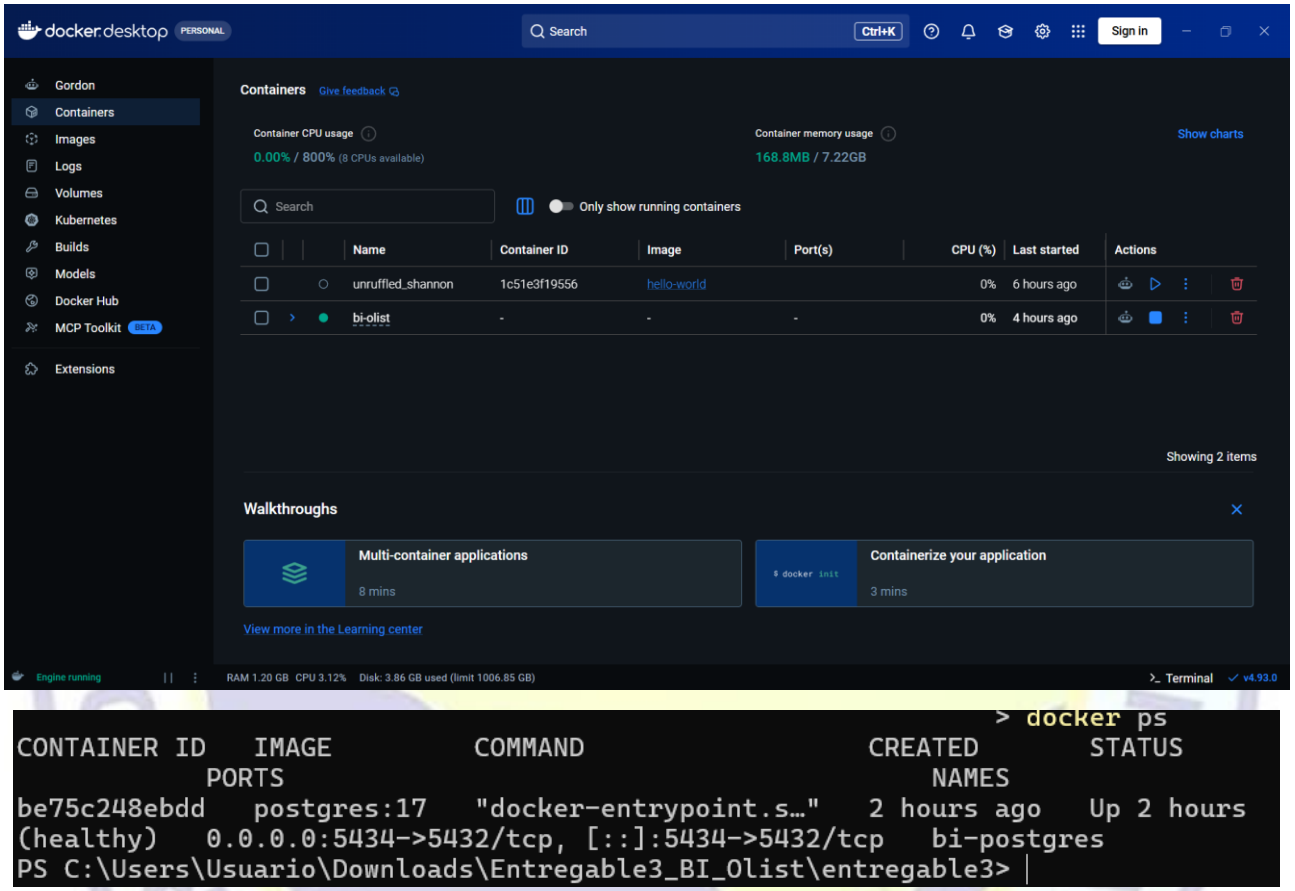


Figura 3. Modelo en pgAdmin (ERD) con las clave|s primarias y foráneas

5. Implementación en PostgreSQL

5.1 Plataforma

PostgreSQL 17 se ejecuta en un contenedor Docker definido en docker-compose.yml, con credenciales y puerto configurables en el archivo .env. La base de datos se llama bi_database y contiene dos esquemas: stg (staging, tablas limpias de paso) y dw (modelo dimensional).




```
                                                    > docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED       STATUS
          PORTS                                        NAMES
be75c248ebdd   postgres:17    "docker-entrypoint.s…"   2 hours ago   Up 2 hours
(healthy)   0.0.0.0:5434->5432/tcp, [::]:5434->5432/tcp   bi-postgres
PS C:\Users\Usuario\Downloads\Entregable3_BI_Olist\entregable3>
```

Figura 4. Contenedor bi-postgres en estado healthy (comando docker ps)

5.2 Scripts de creación y consulta

Script	Función
python/etl_olist.py	Limpieza y transformación con pandas; carga las tablas limpias al esquema stg
sql/01_staging.sql	Crea el esquema stg (tablas de paso)
sql/02_modelo_dimensional.sql	Crea el esquema dw: dimensiones, tabla de hechos, claves primarias y foráneas
sql/03_carga_dimensiones_hechos.sql	Carga las dimensiones y la tabla de hechos desde staging
sql/04_validaciones.sql	Consultas de validación: conteos, integridad, nulos, totales
sql/05_vistas_funciones.sql	Funciones KPI y vistas para el dashboard
sql/06_pruebas_objetos.sql	Pruebas de las vistas y funciones
sql/07_funciones_app.sql	Funciones con filtros para Streamlit (ventas por estado, logística, retraso vs reseña, pagos, vendedores, detalle, variación mensual y listas de filtros)

Script	Función
sql/08_validaciones_extra.sql	Conciliación entre pedidos originales y cargados; ingresos brutos vs válidos
sql/09_pruebas_funciones_app.sql	Pruebas de las funciones de la aplicación
docker/run_pipeline.sh	Orquesta todo el proceso en el orden correcto

5.3 Proceso de carga

Todo el proceso se ejecuta con un solo comando: docker compose run --rm etl. El script docker/run_pipeline.sh realiza, en orden: verificación de los CSV, espera de PostgreSQL, limpieza y carga a staging con Python, creación del modelo, carga de dimensiones y hechos, validaciones, creación de vistas y funciones, y pruebas.

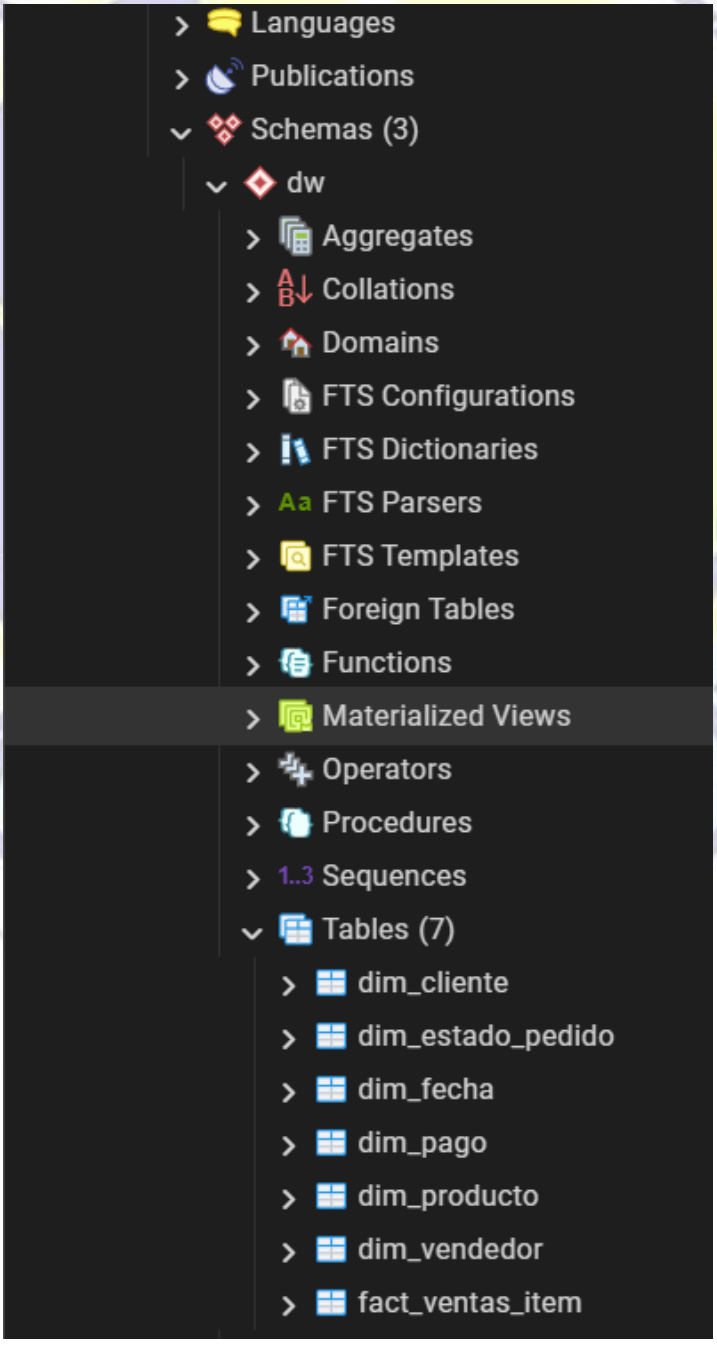


Figura 5. Tablas del esquema dw visibles en pgAdmin

6. Validación de los datos

Las consultas de sql/04_validaciones.sql y sql/08_validaciones_extra.sql demuestran que la información fue cargada correctamente.

Verificación	Resultado
Registros cargados	dim_cliente 95 774 · dim_estado_pedido 8 · dim_fecha 1 461 · dim_pago 25 · dim_producto 32 951 · dim_vendedor 3 095 · fact_ventas_item 112 279
Coherencia staging vs hechos (ítems)	Staging 112 279 = hechos 112 279; diferencia 0
Suma de price: staging vs hechos	R$ 13 541 712.78 en ambos; diferencia 0.00
Integridad de claves	0 duplicados en la clave primaria y 0 claves huérfanas en las 7 relaciones; 14 restricciones (6 PK de dimensiones, 1 PK de hechos y 7 FK) verificadas
Nulos relevantes en llaves	Sin fecha de compra, cliente, producto, vendedor, estado o pago: 0 casos
Nulos esperados	2 406 ítems sin fecha de entrega (pedidos no entregados o cancelados) y 936 sin reseña; son nulos legítimos
Rango y coherencia	0 montos negativos, 0 entregas antes de la compra, 0 calificaciones fuera de 1–5
Pedidos y período	98 353 pedidos con ítems; primera compra 2017-01-05, última 2018-08-29
Pedidos sin ítems	El staging tiene 99 092 pedidos y los hechos 98 353; los 739 de diferencia son pedidos que no tienen ítems asociados y por eso no generan filas de hechos (detalle en la captura de validaciones_extra)
Ingresos válidos vs brutos	Brutos R$ 13 541 712.78; válidos R$ 13 449 529.68 (se excluyen cancelados y no disponibles)

```
       tabla       | filas
-------------------+--------
 dim_cliente       |  95774
 dim_estado_pedido |      8
 dim_fecha         |   1461
 dim_pago          |     25
 dim_producto      |  32951
 dim_vendedor      |   3095
 fact_ventas_item  | 112279
(7 rows)

 items_en_staging | filas_en_hechos | diferencia
------------------+-----------------+------------
           112279 |          112279 |          0
(1 row)

 sin_fecha_compra | sin_fecha_entrega | sin_cliente | sin_producto | sin_vendedor | sin_estado | sin_pago
------------------+-------------------+-------------+--------------+--------------+------------+----------
                0 |                 0 |           0 |            0 |            0 |          0 |        0
(1 row)

 order_id | order_item_id | count
----------+---------------+-------
(0 rows)

 dim | clave | count
-----+-------+-------
(0 rows)
```

```
        tabla         |              restriccion              |                                                  definicion
----------------------+---------------------------------------+---------------------------------------------------------------------------------------------------
 dw.dim_cliente       | dim_cliente_pkey                      | PRIMARY KEY (cliente_key)
 dw.dim_estado_pedido | dim_estado_pedido_pkey                | PRIMARY KEY (estado_key)
 dw.dim_fecha         | dim_fecha_pkey                        | PRIMARY KEY (fecha_key)
 dw.dim_pago          | dim_pago_pkey                         | PRIMARY KEY (pago_key)
 dw.dim_producto      | dim_producto_pkey                     | PRIMARY KEY (producto_key)
 dw.dim_vendedor      | dim_vendedor_pkey                     | PRIMARY KEY (vendedor_key)
 dw.fact_ventas_item  | fact_ventas_item_pkey                 | PRIMARY KEY (order_id, order_item_id)
 dw.fact_ventas_item  | fact_ventas_item_pago_key_fkey        | FOREIGN KEY (pago_key) REFERENCES dw.dim_pago(pago_key)
 dw.fact_ventas_item  | fact_ventas_item_fecha_compra_key_fkey| FOREIGN KEY (fecha_compra_key) REFERENCES dw.dim_fecha(fecha_key)
 dw.fact_ventas_item  | fact_ventas_item_fecha_entrega_key_fkey| FOREIGN KEY (fecha_entrega_key) REFERENCES dw.dim_fecha(fecha_key)
 dw.fact_ventas_item  | fact_ventas_item_cliente_key_fkey     | FOREIGN KEY (cliente_key) REFERENCES dw.dim_cliente(cliente_key)
 dw.fact_ventas_item  | fact_ventas_item_producto_key_fkey    | FOREIGN KEY (producto_key) REFERENCES dw.dim_producto(producto_key)
 dw.fact_ventas_item  | fact_ventas_item_vendedor_key_fkey    | FOREIGN KEY (vendedor_key) REFERENCES dw.dim_vendedor(vendedor_key)
 dw.fact_ventas_item  | fact_ventas_item_estado_key_fkey      | FOREIGN KEY (estado_key) REFERENCES dw.dim_estado_pedido(estado_key)
(14 rows)

 filas  | sin_fecha_entrega | sin_dias_entrega | sin_review | precio_nulo | flete_nulo
--------+-------------------+------------------+------------+-------------+------------
 112279 |              2406 |             2406 |        936 |           0 |          0
(1 row)

 order_status | items  | sin_fecha_entrega
--------------+--------+-------------------
 delivered    | 109880 |                 8
 shipped      |   1174 |              1174
 canceled     |    527 |               526
 processing   |    355 |               355
 invoiced     |    340 |               340
 approved     |      3 |                 3
(6 rows)
```

```
 montos_negativos | entrega_antes_de_compra | score_fuera_de_rango
------------------+-------------------------+----------------------
                0 |                       0 |                    0
(1 row)

 pedidos | items  | ingresos_brutos | flete_total | ticket_promedio_bruto | primera_compra | ultima_compra
---------+--------+-----------------+-------------+-----------------------+----------------+---------------
   98353 | 112279 |     13541712.78 |  2244490.79 |                137.68 | 2017-01-05     | 2018-08-29
(1 row)

 price_staging | price_hechos | diferencia
---------------+--------------+------------
   13541712.78 |  13541712.78 |       0.00
(1 row)

 anio_mes | pedidos |  ingresos
----------+---------+------------
 2017-01  |     787 |  120098.27
 2017-02  |    1718 |  244959.35
 2017-03  |    2617 |  368341.32
 2017-04  |    2377 |  353842.98
 2017-05  |    3640 |  503159.19
 2017-06  |    3205 |  429916.61
 2017-07  |    3946 |  492287.30
 2017-08  |    4272 |  568245.79
 2017-09  |    4227 |  621415.91
 2017-10  |    4547 |  660179.62
 2017-11  |    7421 | 1003862.14
 2017-12  |    5618 |  742183.79
 2018-01  |    7187 |  945456.29
 2018-02  |    6624 |  837895.43
 2018-03  |    7168 |  981051.06
 2018-04  |    6919 |  993592.98
 2018-05  |    6833 |  992871.75
 2018-06  |    6145 |  863265.53
 2018-07  |    6233 |  878044.27
 2018-08  |    6421 |  848860.10
(20 rows)
```

Figura 6. Resultado de las consultas de validación (validaciones.txt o pgAdmin)

```
 pedidos_staging | pedidos_en_hechos | pedidos_sin_items
-----------------+-------------------+-------------------
           99092 |             98353 |               739
(1 row)

 order_status | pedidos_sin_items
--------------+-------------------
 unavailable  |               602
 canceled     |               132
 created      |                 5
(3 rows)

 ingresos_brutos | ingresos_validos | excluidos_cancelados
-----------------+------------------+----------------------
     13541712.78 |      13449529.68 |             92183.10
(1 row)

 desde_funcion | desde_hechos
---------------+--------------
   13449529.68 |  13449529.68
(1 row)
```

Figura 7. Conciliación de pedidos en staging vs hechos (validaciones_extra)

7. Vistas y funciones SQL

Los objetos SQL alimentan directamente el dashboard. Las funciones reciben los filtros como parámetros (NULL significa sin filtro).

Objeto SQL	Tipo	Devuelve
fn_base_pedidos(desde, hasta, categoría, estado)	Función de tabla	Una fila por pedido con ingresos, flete, entrega y reseña; base de los KPI
fn_total_ventas / fn_ticket_promedio / fn_tasa_recompra	Función escalar	KPI de ventas
fn_tasa_entrega_a_tiempo / fn_tiempo_promedio_entrega / fn_calificacion_promedio	Función escalar	KPI de logística y satisfacción
fn_ventas_mensuales / fn_top_categorias / fn_ranking_vendedores	Función de tabla	Series y rankings (Entregable 2)
fn_ventas_estado / fn_logistica_estado / fn_retraso_vs_resena / fn_metodos_pago	Función de tabla	Tablas y gráficos con filtros (Entregable 3)
fn_variacion_mensual / fn_variacion_ultimo_mes	Función	Variación de ingresos frente al mes anterior
fn_vendedores_filtrado / fn_detalle_ventas	Función de tabla	Ranking de vendedores y detalle de ítems con todos los filtros
fn_lista_categorias / fn_lista_estados_cliente / fn_rango_fechas	Función	Valores para poblar los filtros de la aplicación
v_pedidos, v_ventas_mensual, v_ventas_categoria, v_ventas_estado, v_logistica_estado, v_retraso_vs_resena, v_metodos_pago, v_detalle_ventas	Vistas	Consultas sin parámetros sobre todo el período

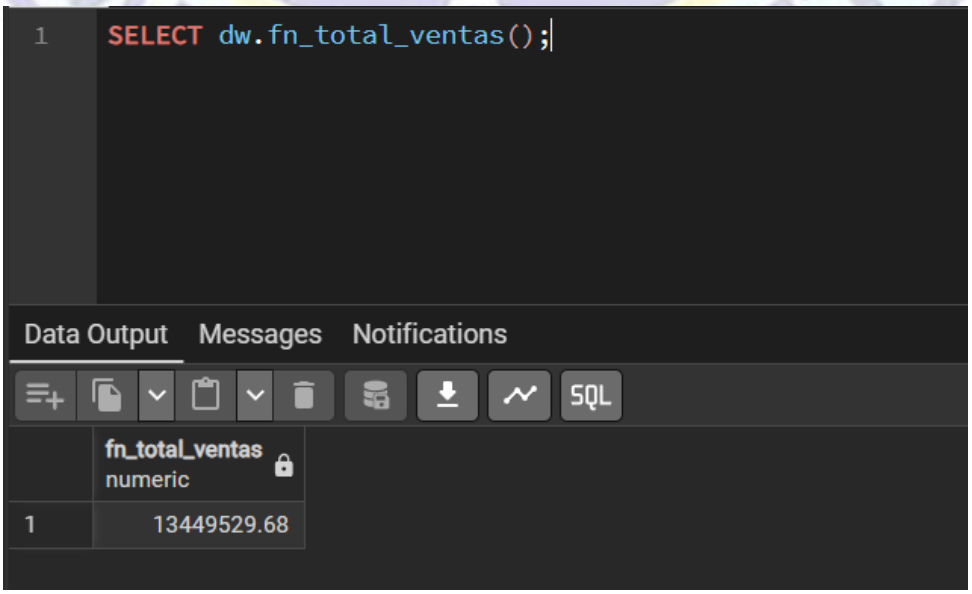


Figura 8. Ejecución de una función en pgAdmin, por ejemplo SELECT dw.fn_total_ventas();

8. Mockup de la aplicación

El dashboard se organiza en tres páginas, con una barra lateral de filtros común (rango de fechas, categoría y estado del cliente). Cada página tiene 3 tarjetas de KPI, 2 gráficos y una tabla. Cada componente indica el objeto SQL que lo alimenta.

- **Página 1 · Ventas:** ingresos, ticket promedio y recompra; evolución mensual y top de categorías; ventas por estado.
- **Página 2 · Logística y satisfacción:** entrega a tiempo, tiempo de entrega y calificación; entrega a tiempo por estado y calificación según retraso; detalle de ítems.
- **Página 3 · Vendedores y pagos:** ingresos, calificación y recompra; top de vendedores y pedidos por tipo de pago; ranking de vendedores.



Figura 9. Mockup página 1 · Ventas (corregido)

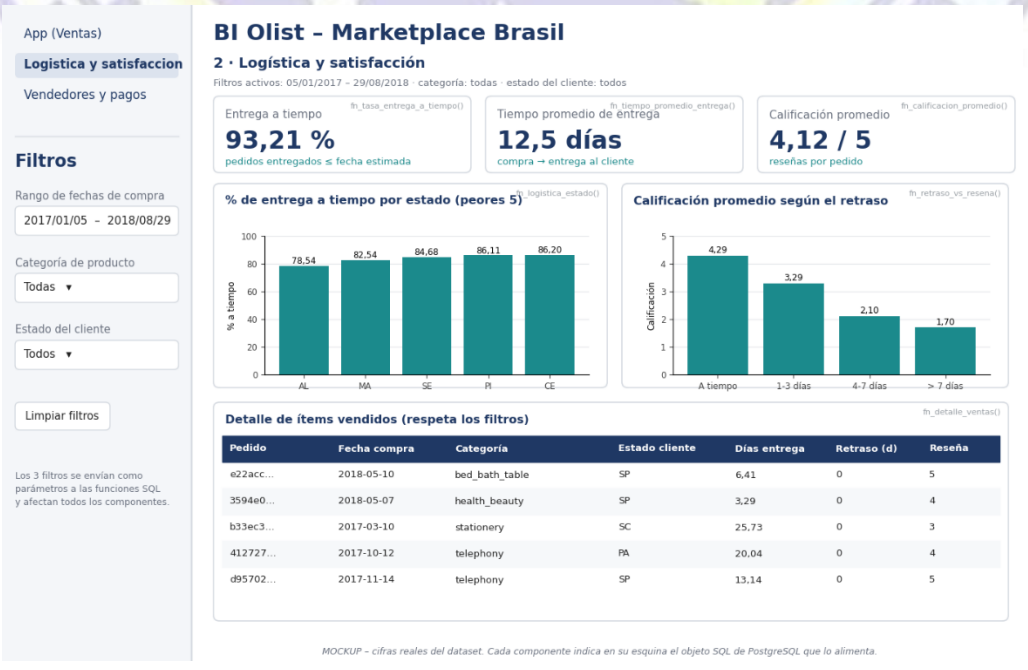


Figura 10. Mockup página 2 · Logística y satisfacción (corregido)



Figura 11. Mockup página 3 · Vendedores y pagos (corregido)

9. Matriz de trazabilidad

Cada componente del dashboard se relaciona con la pregunta o KPI que responde, los filtros que lo afectan, los datos utilizados y el objeto SQL de PostgreSQL que lo alimenta.

Componente	Pregunta/KPI	Filtros	Datos utilizados	Vista / Función SQL
Pág.1 · Tarjeta K1: Ingresos totales (+ variación vs. mes anterior)	P1 · Ingresos por ventas	Fecha, categoría, estado cliente	Fact + dim_fecha, dim_producto, dim_cliente, dim_estado_pedido	dw.fn_total_ventas(); dw.fn_variacion_ultimo_mes()
Pág.1 · Tarjeta K2: Ticket promedio	P1, P5 · Ticket promedio (AOV)	Fecha, categoría, estado cliente	Fact (pedidos únicos) + dimensiones	dw.fn_ticket_promedio()
Pág.1 · Tarjeta K3: Tasa de recompra	P5 · Tasa de recompra	Fecha, categoría, estado cliente	Fact + dim_cliente (customer_unique_id)	dw.fn_tasa_recompra()
Pág.1 · Gráfico G1: Ingresos mensuales (línea)	P1 · Evolución de ventas por mes	Fecha, categoría, estado cliente	Fact + dim_fecha	dw.fn_ventas_mensuales()
Pág.1 · Gráfico G2: Top 10 categorías (barras)	P1 · Categorías con más ingresos	Fecha, estado cliente	Fact + dim_producto	dw.fn_top_categorias()
Pág.1 · Tabla T1: Ventas por región y estado	P1 · Estados que generan más ingresos	Fecha, categoría, estado cliente	Fact + dim_cliente	dw.fn_ventas_estado()
Pág.2 · Tarjeta K4: Entrega a tiempo	P2, P3 · Tasa de entrega a tiempo	Fecha, categoría, estado cliente	Fact (entrega_a_tiempo) + dimensiones	dw.fn_tasa_entrega_a_tiempo()
Pág.2 · Tarjeta K5: Tiempo promedio de entrega	P2 · Tiempo promedio de entrega	Fecha, categoría, estado cliente	Fact (dias_entrega) + dimensiones	dw.fn_tiempo_promedio_entrega()
Pág.2 · Tarjeta K6: Calificación promedio	P3, P4 · Calificación promedio	Fecha, categoría, estado cliente	Fact (review_score) + dimensiones	dw.fn_calificacion_promedio()
Pág.2 · Gráfico G3: % a tiempo por estado	P2 · Estados con más retrasos	Fecha, categoría, estado cliente	Fact + dim_cliente	dw.fn_logistica_estado()
Pág.2 · Gráfico G4: Calificación según retraso	P3 · Efecto del retraso en la satisfacción	Fecha, categoría, estado cliente	Fact (dias_retraso, review_score)	dw.fn_retraso_vs_resena()
Pág.2 · Tabla T2: Detalle de ítems	P2, P3 · Drill-down	Fecha, categoría, estado cliente	Fact + todas las dimensiones	dw.fn_detalle_ventas()

Componente	Pregunta/KPI	Filtros	Datos utilizados	Vista / Función SQL
Pág.3 · Gráfico G5: Pedidos por tipo de pago	P5 · Medios de pago preferidos	Fecha, categoría, estado cliente	Fact + dim_pago	dw.fn_metodos_pago()
Pág.3 · Gráfico G6: Top vendedores	P4 · Vendedores que concentran ingresos	Fecha, categoría, estado cliente	Fact + dim_vendedor	dw.fn_vendedores_filtrado()
Pág.3 · Tabla T3: Ranking de vendedores	P4 · Ingresos y reputación	Fecha, categoría, estado cliente	Fact + dim_vendedor	dw.fn_vendedores_filtrado()
Filtros (fecha, categoría, estado cliente)	Todos	—	dim_fecha, dim_producto, dim_cliente	dw.fn_rango_fechas(); dw.fn_lista_categorias(); dw.fn_lista_estados_cliente()

10. Aplicación Python + Streamlit

10.1 Arquitectura

La aplicación está en la carpeta app/: db.py gestiona la conexión a PostgreSQL y ejecuta consultas parametrizadas; filtros.py dibuja los filtros y los comparte entre páginas; app.py y las dos páginas de la carpeta pages/ construyen los KPI, gráficos y tablas. Se ejecuta con streamlit run app.py.

10.2 Cumplimiento de los requisitos

Requisito	Cómo se cumple
Conexión real con PostgreSQL	app/db.py usa psycopg2 y lee host, puerto y credenciales del archivo .env
2 vistas o secciones analíticas	3 páginas: 1 Ventas · 2 Logística y satisfacción · 3 Vendedores y pagos
3 KPI	3 tarjetas por página (6 KPI distintos en total)
3 filtros interactivos	Rango de fechas, categoría de producto y estado del cliente; se conservan al cambiar de página
2 gráficos	2 gráficos por página (6 en total) con Plotly
1 tabla de detalle	Tabla de ventas por estado, detalle de ítems (con control de filas) y ranking de vendedores
Consultas que respondan las preguntas	Cada componente llama a una función SQL (ver matriz de trazabilidad)
Restricción: no leer el CSV	La aplicación solo ejecuta SELECT sobre funciones de PostgreSQL; no contiene ninguna lectura de archivos CSV
Filtros que afectan los resultados	Cada filtro se envía como parámetro a las funciones SQL; al cambiarlo se recalculan KPI, gráficos y tablas

10.3 Evidencias de funcionamiento

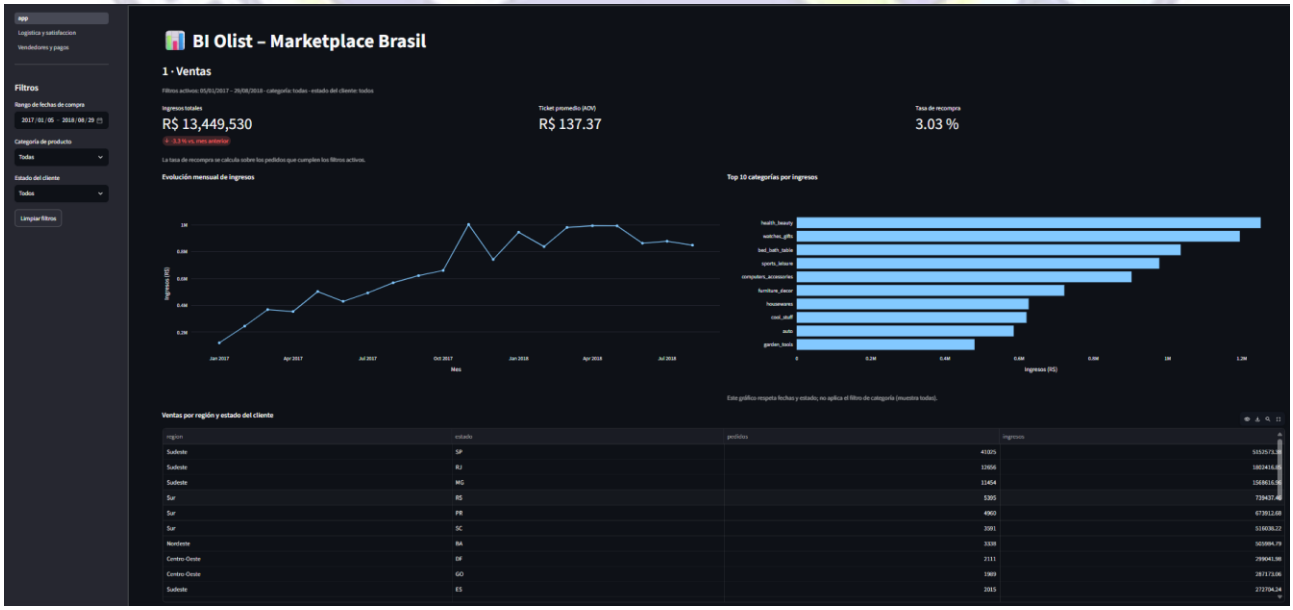


Figura 12. Página 1 · Ventas sin filtros



Figura 13. Página 1 · Ventas con filtros aplicados (año 2017, estado SP)

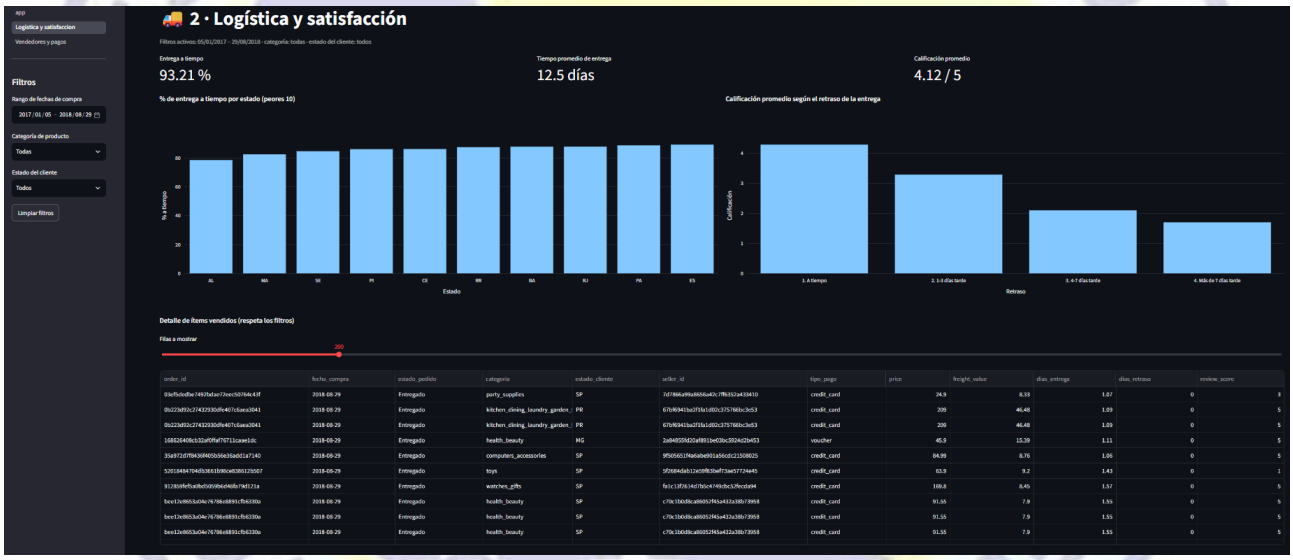


Figura 14. Página 2 · Logística y satisfacción



Figura 15. Página 3 · Vendedores y pagos

10.4 Verificación de coherencia entre la aplicación y PostgreSQL

Con los filtros año 2017 y estado SP, la aplicación muestra ingresos de R$ 2.192.397 La misma consulta ejecutada directamente en PostgreSQL devuelve el mismo valor, lo que demuestra que los filtros llegan a las consultas y que los datos provienen de la base.

```
SELECT dw.fn_total_ventas('2017-01-01', '2017-12-31', NULL, 'SP');
-- resultado: 2192396.73
```

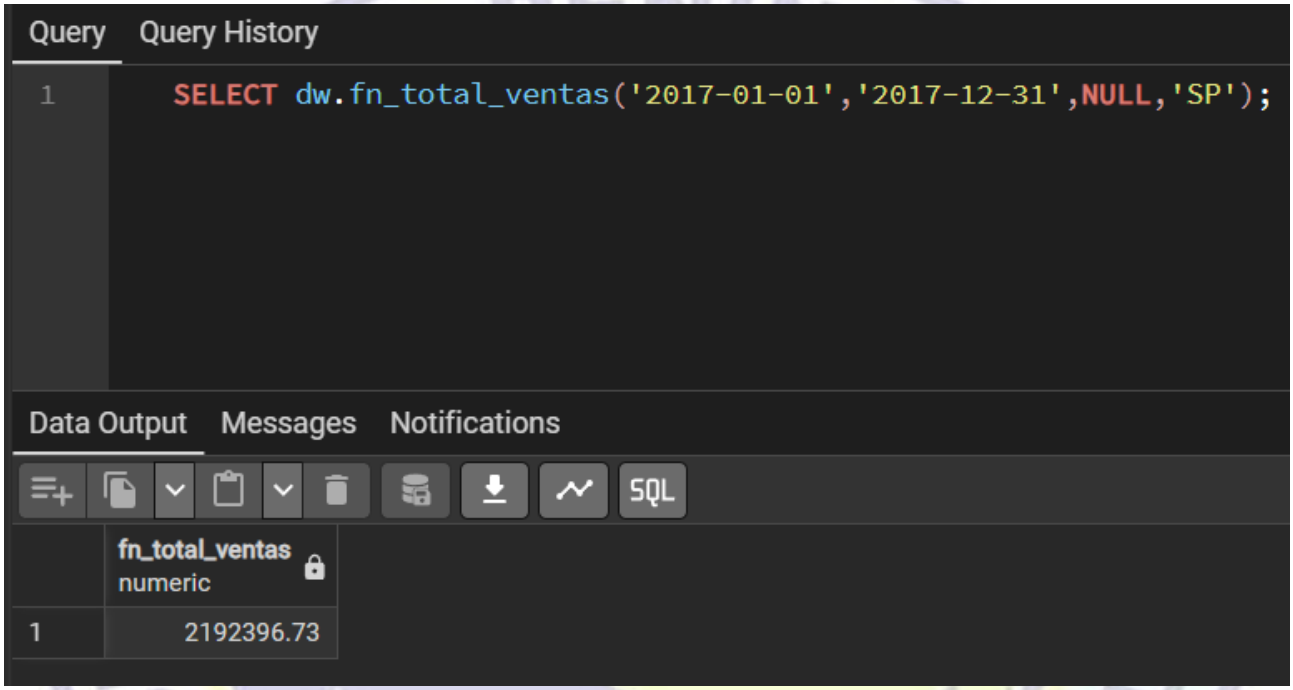


Figura 16. Consulta en pgAdmin con el mismo resultado que la aplicación

11. Código fuente Python

El código completo está en el repositorio. Los archivos principales son:

Archivo	Descripción
python/etl_olist.py	ETL: lee los 9 CSV, limpia y transforma, y carga el esquema stg
app/db.py	Conexión a PostgreSQL con psycopg2 y función consulta() para ejecutar SELECT parametrizados
app/filtros.py	Filtros de fecha, categoría y estado; arma los parámetros de las funciones SQL
app/app.py	Página 1 · Ventas
app/pages/2_Logistica_y_satisfaccion.py	Página 2 · Logística y satisfacción
app/pages/3_Vendedores_y_pagos.py	Página 3 · Vendedores y pagos

Fragmento representativo: la aplicación consulta PostgreSQL pasando los filtros como parámetros.

```
k = consulta(f"""SELECT dw.fn_total_ventas({F}) AS ingresos,
                        dw.fn_ticket_promedio({F}) AS ticket,
                        dw.fn_tasa_recompra({F}) AS recompra""", a * 3)
# F = "%s::date, %s::date, %s::text, %s::char(2)"  -> desde, hasta, categoría, estado
```

12. Instrucciones de ejecución (README)

Requisitos: Docker Desktop, Python 3.10 o superior y los 9 CSV de Kaggle en data/raw/.

```
# 1) (opcional) cambiar el puerto: copiar .env.example como .env y editar DB_PORT
# 2) Base de datos, limpieza, modelo, carga, validaciones y funciones
docker compose run --rm etl

# 3) Aplicación
cd app
pip install -r requirements.txt
streamlit run app.py     # http://localhost:8501
```

Conexión desde pgAdmin: host 127.0.0.1, puerto según DB_PORT, base bi_database, usuario bi_user. El README.md del repositorio incluye una tabla de problemas frecuentes.

13. Evidencias de uso de Inteligencia Artificial

Se utilizó IA generativa como apoyo para aprender, diseñar consultas y resolver errores. En cada caso el código fue ejecutado, comprobado y adaptado. Se documentan tres usos significativos.

Uso 1

Aspecto	Descripción
Problema que se quería resolver	pgAdmin no podía conectarse a la base de datos del proyecto y mostraba “la autenticación password falló para el usuario bi_user”.
Prompt o consulta realizada	“Unable to connect to server: connection failed: connection to server at 127.0.0.1, port 5432 failed: FATAL: la autenticación password falló para el usuario bi_user”. Después, al ver en docker

Aspecto	Descripción
	ps el puerto 0.0.0.0:5432->5432, pregunté qué hacer, y también comenté que tenía otro pgAdmin instalado.
Solución propuesta por la IA	revisar con docker ps qué contenedor y puerto estaban activos; recrear la base con docker compose down -v y docker compose run --rm etl; y, por si otro PostgreSQL ocupaba el puerto 5432, crear un archivo.env con DB_PORT=5434. También aclaró que pgAdmin es solo el visor y que el servidor PostgreSQL es lo que ocupa el puerto.
Qué se utilizó o modificó	ejecuté docker compose down -v y volví a cargar los datos con docker compose run --rm etl. [confirma si además cambiaste el puerto en .env y a cuál] Usé 127.0.0.1 como host en pgAdmin.
Cómo se comprobó que era correcta	docker ps mostró el contenedor bi-postgres en estado *healthy*, y la aplicación se conectó y mostró datos reales (ingresos de R$ 13.449.530).

Uso 2

Aspecto	Descripción
Problema que se quería resolver	necesitaba revisar mi Entregable 2 antes de construir la aplicación, y que los filtros de la app afectaran realmente a los resultados.
Prompt o consulta realizada	subí el ZIP de mi Fase 2 y las instrucciones del Entregable 3, y pedí que la IA analizara el proyecto. Después pedí un paquete completo y listo para ejecutar con la aplicación de Streamlit.
Solución propuesta por la IA	detectó que varias vistas de la Fase 2 no aceptaban filtros, que los mockups tenían cifras inventadas, que la tarjeta “vs. mes anterior” no tenía un objeto SQL y que había 739 pedidos de diferencia entre staging y hechos sin explicar. Propuso funciones SQL parametrizadas (fn_ventas_estado, fn_logistica_estado, fn_variacion_mensual, fn_vendedores_filtrado, entre otras) y una aplicación de 3 páginas.
Qué se utilizó o modificó	usé los scripts 07_funciones_app.sql, 08_validaciones_extra.sql y la carpeta app/. Corregí los mockups con cifras reales y mantuve mi modelo estrella, que no se modificó. [confirma si cambiaste algo más]
Cómo se comprobó que era correcta	ejecuté las funciones en pgAdmin y comparé: con año 2017 y estado SP, la app muestra R$ 2.192.397 y la consulta SELECT dw.fn_total_ventas('2017-01-01','2017-12-31',NULL,'SP'); devuelve 2192396.73. Además, al cambiar un filtro cambian los KPI, los gráficos y las tablas.

Uso 3

Aspecto	Descripción
Problema que se quería resolver	uería ver solo las ventas de 2017, pero el filtro de fechas se quedaba incompleto (mostraba 2017 / mm / dd) y la app no actualizaba los datos.
Prompt o consulta realizada	“Vamos por el paso 7, ¿cómo filtro el año 2017? Porque mira, me sale así”, con una captura de la pantalla.
Solución propuesta por la IA	explicó que el filtro se escribe en orden año/mes/día, que deben completarse las dos fechas (inicio y fin) para que la app se actualice, y que el inicio debía ser 05/01/2017 porque la primera compra de la base es de esa fecha. También recomendó desactivar la traducción automática del navegador, porque cambiaba nombres reales de categorías.
Qué se utilizó o modificó	puse el rango 05/01/2017 – 31/12/2017 y el estado SP, y desactivé la traducción del navegador antes de tomar las capturas.
Cómo se comprobó que era correcta	la app mostró ingresos de R$ 2.192.397 con esos filtros, coincidiendo con el resultado de la consulta SQL directa en la base de datos (2192396.73).

14. Resultados y decisiones

Con los datos cargados, la solución permite responder las preguntas de negocio. Algunos resultados:

- Los ingresos válidos del período suman R$ 13 449 529.68 con un ticket promedio de R$ 137.37.
- El mes con mayores ingresos fue noviembre de 2017 (R$ 1 003 862.14); desde 2018 los ingresos mensuales se mantienen entre R$ 838 mil y R$ 994 mil.
- Las categorías con más ingresos son health_beauty (R$ 1 251 145.54), watches_gifts (R$ 1 194 824.97) y bed_bath_table (R$ 1 035 485.07).
- El estado de São Paulo (SP) concentra R$ 5 152 573, cerca del 38 % de los ingresos.
- El 93.21 % de los pedidos entregados llega en la fecha estimada o antes, con un tiempo promedio de 12.5 días.
- La tasa de recompra es baja (3.03 %), lo que indica poco retorno de clientes.

Efecto del retraso en la satisfacción: 1,70 para los que llegan con más de 7 días de retraso.

Decisiones que se pueden apoyar

- Comercial: reforzar promociones en las categorías líderes y en los estados con mayor demanda.
- Logística: revisar plazos estimados y rutas en los estados con menor porcentaje de entrega a tiempo.
- Vendedores: premiar a los de mayores ingresos y buena calificación, y dar seguimiento a los de alto retraso.
- Marketing: diseñar campañas de fidelización para elevar la recompra.

15. Conclusiones

¿Qué voy a analizar, con qué datos y para qué servirán los resultados?

Se analiza el desempeño comercial, logístico y de satisfacción del marketplace Olist, con el Brazilian E-Commerce Public Dataset (aproximadamente 100 mil pedidos y 112 mil ítems entre 2017 y 2018). Los resultados sirven para que las áreas comerciales, de logística y de gestión de vendedores decidan qué categorías y regiones impulsar, dónde mejorar los tiempos de entrega y a qué vendedores premiar o corregir.

¿Cómo se almacena, consulta y presenta la información?

Los datos se limpian con Python, se almacenan en un modelo dimensional en PostgreSQL, se consultan mediante funciones SQL parametrizadas y se presentan en un dashboard de Streamlit con filtros interactivos.

- El modelo dimensional con granularidad de ítem de pedido soporta los 6 KPI definidos.
- Las validaciones demuestran que la carga es completa y consistente con los datos originales.
- La aplicación obtiene todos sus datos de PostgreSQL y sus filtros afectan realmente a los resultados.

Enlaces:

- dataset:

  https://www.kaggle.com/datasets/olistbr/brazilian-ecommerc
- repositorio:

  Andyquimi/bi-olist-entregable3: Solución de BI con PostgreSQL, Python y Streamlit (dataset Olist)
- video (DRIVE):
  https://drive.google.com/file/d/1a5yoY9rqpREGdJgoE0TCxN9wZE7lzNjU/view?usp=sharing